

Diario di tirocinio

Giorgio Lai

Giorno 1 (28/08/2026)

Attività svolta

Ho iniziato il tirocinio seguendo un processo inverso. Invece di partire dall'implementazione e dall'architettura del sistema, ho cominciato a ragionare sugli esperimenti e sui risultati che il lavoro dovrebbe permettere di ottenere.

Ho svolto alcuni esercizi per comprendere come costruire una domanda controllata. Come esempio ho considerato un utente che porta avanti lo stesso progetto attraverso più conversazioni. Il sistema dovrebbe ricordare lo scopo del progetto, le decisioni ancora valide, le attività completate e quelle rimaste in sospeso, senza chiedere all'utente di ripetere ogni volta il contesto.

Con l'aiuto di ChatGPT ho preparato un primo documento che definisce:

- la domanda di ricerca;
- le condizioni C0, C1 e C2;
- la variabile modificata e le variabili controllate;
- gli aspetti esclusi dal progetto;
- l'unità sperimentale;
- la struttura dell'oracle;
- le definizioni delle metriche;
- la classificazione degli errori;
- i risultati qualitativi attesi;
- la futura tabella dei risultati;
- i criteri per iniziare il pilot.

Cosa ho compreso

Ho compreso che una risposta sbagliata può avere cause differenti. L'informazione potrebbe trovarsi in un archivio non accessibile, potrebbe essere accessibile ma non recuperata correttamente, oppure potrebbe essere fornita al modello ma utilizzata male nella risposta.

È quindi necessario distinguere:

- problemi di accesso o frammentazione;
- problemi di retrieval e ranking;
- problemi di lettura e generazione.

Ho inoltre iniziato a distinguere tre possibili condizioni:

- C0: assenza di memoria;
- C1: memoria limitata alla sessione corrente;
- C2: memoria condivisa tra le sessioni dello stesso utente.

Difficoltà incontrate

Inizialmente tendevo a confondere la mancata accessibilità di un'informazione con un errore di retrieval. Attraverso gli esempi ho capito che il retrieval non può recuperare un'informazione presente in un archivio che il sistema non è autorizzato a consultare.

Giorno 2 (29/08/2026)

Attività svolta

Ho iniziato a progettare il primo esperimento. Come dominio generale ho scelto la cybersecurity. In particolare, ho deciso di rappresentare la gestione di un incidente distribuita tra più conversazioni.

Con l'aiuto di ChatGPT ho preparato una narrazione contenente:

1. una situazione iniziale;
2. un primo allarme;
3. un'ipotesi iniziale;
4. nuove evidenze;
5. una correzione dell'ipotesi;
6. alcune azioni completate;
7. alcune verifiche ancora in sospeso;
8. alcune informazioni mai fornite.

Ho quindi costruito il primo scenario, diviso in quattro conversazioni. Successivamente ho preparato sette domande per controllare quali risposte un'AI può fornire senza memoria, usando soltanto l'ultima conversazione oppure utilizzando anche le conversazioni precedenti. Prima di eseguire le prove ho definito nell'oracle le risposte attese e le informazioni necessarie per ciascuna domanda.

Ho poi eseguito un dry run su Claude. Per ognuna delle sette domande ho provato i tre casi: senza memoria, con memoria locale e con memoria condivisa. Ho quindi eseguito 21 prove in chat separate.

Nel caso senza memoria, Claude riceveva soltanto la domanda. Con la memoria locale riceveva la domanda e l'ultima sessione. Con la memoria condivisa riceveva invece la domanda e tutte le quattro sessioni dello scenario. In ogni prova ho chiesto a Claude di usare soltanto le informazioni fornite e di non fare supposizioni.

Il dry run ha permesso di individuare un errore nell'oracle della domanda Q4. L'ultima sessione conteneva già le due verifiche richieste, quindi la domanda poteva essere risolta anche con la memoria locale. Ho corretto l'oracle e la matrice di raggiungibilità. Dopo questa correzione, 20 comportamenti su 21 sono risultati appropriati. L'unico errore rimasto era dovuto al modello, che in una risposta ha confuso gli obiettivi iniziali con le attività ancora in sospeso.

Successivamente ho preparato un secondo scenario, dedicato al recupero password di una piattaforma inventata. Anche questo scenario è diviso in quattro conversazioni e contiene sette domande con le relative risposte attese. Il secondo scenario è stato costruito in modo da aumentare il numero di domande risolvibili con la sola memoria locale.

Ho eseguito anche per il secondo scenario le 21 prove del dry run. In questo caso 19 comportamenti su 21 sono risultati appropriati. Nei due casi sbagliati, il modello ha indicato l'attività ancora aperta invece di quella già completata. Le domande non sono state modificate, perché le informazioni necessarie erano presenti e chiare.

Cosa ho compreso

In totale ho preparato due scenari, quattordici domande e quarantadue prove. Queste prove sono servite a controllare la qualità degli scenari, delle domande e delle risposte attese.

Ho compreso che non bisogna modificare automaticamente una domanda ogni volta che il modello sbaglia. Prima è necessario stabilire se l'errore dipende dalla domanda, dall'oracle oppure dal modello.

I risultati ottenuti non rappresentano ancora l'esperimento finale, perché i contesti sono stati forniti manualmente e il sistema di retrieval non è stato ancora implementato.

Giorno 3 (30/08/2026)

Attività svolta

Ripassando il lavoro svolto, mi sono reso conto che dai paper non ho preso soltanto idee sulle possibili architetture, ma anche molte indicazioni su come progettare l'esperimento. In particolare, ho ripreso il confronto tra condizioni controllate, gli scenari su più sessioni, l'uso di domande con evidenze e risposte attese e la separazione tra accesso alla memoria, recupero delle informazioni e risposta del modello. Ho poi adattato questi elementi alla mia domanda di ricerca, mantenendo l'esperimento piccolo e comprensibile.

Riferimenti bibliografici utilizzati

Le idee sperimentali provengono soprattutto dai seguenti paper.

1. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. È la fonte metodologica principale, da cui ho ripreso:
 - la separazione tra indexing, retrieval e reading;
 - le evidenze annotate;
 - le domande multi-sessione;
 - l'aggiornamento delle informazioni;
 - l'astensione quando la risposta non è presente;
 - la valutazione separata di retrieval e risposta.
2. *LoCoMo: Evaluating Very Long-Term Conversational Memory of LLM Agents*. Da questo lavoro ho ripreso:
 - gli scenari conversazionali sintetici;
 - le informazioni distribuite su più sessioni;
 - le domande single-hop, multi-hop e temporali;
 - il confronto tra diverse possibilità di accesso alla memoria;
 - la presenza di domande non rispondibili.
3. *A Survey on the Memory Mechanism of Large Language Model Based Agents*. Mi ha fornito la struttura generale:
 - la distinzione tra scrittura, gestione e lettura della memoria;
 - la separazione tra metriche del retrieval e risultato finale;
 - la necessità di mantenere fissi modello, prompt, dataset e task;
 - l'utilizzo di baseline senza memoria.
4. *Personalization of Large Language Models: A Survey*. Da questo lavoro ho ripreso soprattutto:
 - il confronto controllato tra diversi livelli di memoria;
 - la memoria associata allo stesso utente;
 - la distinzione tra valutazione diretta della risposta e valutazione del sistema complessivo;

- l'attenzione a privacy, informazioni obsolete e personalizzazione.
5. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. In misura secondaria, ha contribuito alle idee su:
- le informazioni aggiornate che sostituiscono quelle precedenti;
 - il controllo delle contraddizioni e delle informazioni obsolete;
 - la valutazione per categoria di domanda;
 - i token e la latenza come misure aggiuntive.

I paper realmente centrali per l'esperimento sono quindi LongMemEval, LoCoMo e la survey sui meccanismi di memoria.

Scelta dell'architettura

Dopo aver definito l'esperimento sono passato alla scelta dell'architettura. Per il primo esperimento ho stabilito:

- architettura principale: Turn-level RAG;
- condizioni: C0, C1 e C2;
- controllo diagnostico: full history, per verificare il comportamento del modello senza errori di retrieval;
- unità di memoria: singolo messaggio con identificatori di scenario, sessione e messaggio;
- stesso retriever: identico in C1 e C2; cambia soltanto il corpus accessibile;
- valutazione: raggiungibilità, evidenze recuperate, risposta, astensione, informazioni obsolete e affermazioni non supportate.

Successivamente ho fatto trasformare a Claude i due scenari in file JSON, assegnando un identificatore ai messaggi, alle domande e alle evidenze. Claude ha anche creato un validatore e alcuni test automatici. Infine ho controllato con Codex che la conversione rispettasse i documenti originali e che tutti i test venissero superati. Il runner con il retrieval non è stato ancora realizzato.

Riepilogo del lavoro svolto

Ho poi deciso di fermarmi per fare un riepilogo, perché voglio prima di tutto capire che cosa sto facendo, invece di avere il progetto già come esperimento finale.

0. Ho applicato un processo inverso: sono partito dagli esperimenti, per poi decidere l'architettura e infine trarre delle conclusioni.
1. **Domanda di ricerca.** Sono partito dalla seguente domanda, che ho intenzione di imparare a memoria: quanto influisce il perimetro della memoria accessibile sulla capacità di un LLM di continuare correttamente un'attività distribuita tra più conversazioni, ossia tra più chat?
Non sto cercando il modello migliore, il retriever migliore, la migliore architettura di memoria in assoluto o la competenza del modello in un dominio.
2. **Variabile modificata.** La variabile che ho deciso di cambiare è quali chat sono accessibili al sistema. Ho tre casi:
 - nessuna memoria: il modello riceve soltanto la domanda;
 - memoria locale: il modello può cercare soltanto nell'ultima sessione (nel nostro caso la sessione 4);
 - memoria condivisa: il modello può cercare in tutte le sessioni.

In ogni caso le altre condizioni devono rimanere uguali (prompt, modello, criteri di valutazione e così via), perché questo serve a poter affermare che le eventuali differenze dipendono soltanto dal perimetro della memoria.

3. **Definizione di una singola prova.** Una prova è composta da uno scenario, una domanda, uno dei tre casi e un oracle. Ogni domanda deve quindi essere eseguita separatamente: la risposta a una domanda non deve diventare memoria per quella successiva.
4. **Dominio di riferimento.** Ho scelto la cybersecurity, utilizzata come contesto per costruire scenari sintetici. Gli scenari sono due: Asteria Docs, che riguarda un accesso anomalo tramite `svc-reporting`, e Lumen Market, che riguarda cambi di password non autorizzati. Nel primo scenario viene formulata un'ipotesi iniziale sulla password; l'ipotesi viene poi superata dalle evidenze sul token `reporting-v1`; viene attivato `reporting-v2`; viene completata l'azione di contenimento e rimangono due verifiche, quindi la sessione 4 stabilisce quale verifica è prioritaria. Nel secondo scenario la decisione iniziale è usare un CAPTCHA, ma le evidenze mostrano che il CAPTCHA non risolverebbe il problema, quindi viene scelta la regola del token monouso valido 15 minuti. Il test mobile viene completato, mentre il test dell'email consegnata in ritardo rimane pendente, quindi viene fissata una riunione di verifica.
5. **Tipi di domande.** Ho preparato sette tipi di domande:
 - scopo o obiettivi;
 - decisione aggiornata e informazioni obsolete;
 - attività completata;
 - attività pendente;
 - informazione presente nella sessione locale;
 - collegamento tra sessioni diverse;
 - informazione mai fornita (per esempio, nel primo scenario, chi è il colpevole del problema).In questo modo non controllo soltanto se il modello “ricorda qualcosa”, ma diverse capacità di memoria.
6. **Oracle manuale.** Ho costruito un oracle manuale, cioè la soluzione di riferimento preparata da una persona prima di eseguire l'esperimento. Non contiene soltanto la risposta corretta: per ogni domanda definisce la risposta attesa, i fatti obbligatori, i messaggi che costituiscono le evidenze, le informazioni obsolete da non utilizzare, le formulazioni equivalenti accettabili, le condizioni nelle quali la risposta è raggiungibile e il comportamento corretto quando l'informazione manca.
7. **Raggiungibilità.** Ho definito la raggiungibilità, che non indica se il modello risponde bene, ma soltanto se le informazioni necessarie sono presenti nel perimetro accessibile.
8. **Cause degli errori.** Ho separato le possibili cause degli errori usando le metriche: l'evidenza è accessibile? Il retriever la recupera? Il modello la usa correttamente?
9. **Dry run manuale.** Per ogni caso (senza memoria, memoria locale e memoria condivisa) ho inviato dei prompt a un LLM (Claude, in questo caso), perché servivano a verificare le domande, l'oracle e il comportamento del modello.
10. Il pilot mi ha permesso di distinguere gli errori del benchmark da quelli del modello: ho corretto l'oracle di Q4 perché era sbagliato, mentre ho mantenuto invariate le domande nei tre casi in cui era il modello a rispondere male.
11. Finora ho dimostrato che i tre casi producono perimetri realmente diversi, che le domande distinguono informazioni locali, distribuite e assenti, che l'oracle permette di riconoscere un difetto del benchmark, che il modello può sbagliare anche quando riceve le evidenze e che il benchmark è utilizzabile per costruire il sistema automatico.
12. **Scelta dell'architettura.** Dopo aver confrontato le architetture dei paper ho deciso di usare Turn-level RAG come architettura principale, full history come controllo diagnostico ed eventualmente Fact-based RAG o Mem0-lite soltanto come futuro secondo esperimento.
13. **Dati strutturati.** Ho trasformato il benchmark in dati strutturati, chiedendo a Claude di convertire gli scenari in file JSON: in questo modo ho il dataset. Adesso ogni scenario, sessione, messaggio, domanda ed evidenza possiede un identificatore.

Prima prova automatica del retrieval

Ho fatto realizzare una prima prova automatica del retrieval Turn-level RAG. Il programma legge i due scenari in formato JSON e per ognuna delle 14 domande esegue le tre condizioni C0, C1 e C2, quindi in totale svolge 42 prove. Il retrieval utilizza TF-IDF e la similarità del coseno, cioè confronta le parole

della domanda con quelle presenti nei messaggi e assegna un punteggio per individuare i messaggi più pertinenti. Vengono considerati soltanto i messaggi dell'utente perché contengono i fatti autorevoli degli scenari. C0 non contiene messaggi, C1 contiene soltanto la Sessione 4 e C2 contiene tutte le sessioni. Il sistema recupera al massimo due messaggi ($\text{top-}k = 2$) e confronta i loro identificatori con le evidenze previste dall'oracle.

Il risultato è stato 5 successi su 5 domande raggiungibili in C1 e 11 successi su 12 in C2. L'unico errore riguarda la domanda Q6 del secondo scenario: l'informazione era accessibile, ma il retriever non ha recuperato il messaggio della Sessione 2 contenente la regola dei 15 minuti. Questo caso viene quindi considerato un errore di retrieval e non un problema della domanda o dell'oracle. Sono stati aggiunti anche alcuni test per verificare il funzionamento del retrieval. In questa fase il sistema recupera soltanto i messaggi e non utilizza ancora un LLM per generare la risposta.

Preparazione degli input

Ho preparato automaticamente gli input che verranno dati al modello. Per ogni domanda ho costruito quattro versioni: senza memoria, con la memoria locale, con la memoria condivisa e con tutto lo storico. In totale sono stati creati 56 input.

Gli input di C1 e C2 utilizzano i messaggi scelti dal retrieval precedente, senza correggere gli eventuali errori. Per esempio, nel caso SC02-Q6, C2 continua a non recuperare la regola dei 15 minuti, mentre questa informazione compare usando tutto lo storico.

Negli input non sono presenti le risposte corrette dell'oracle, perché il modello non deve conoscerle. Per ora ho soltanto preparato ciò che verrà fornito al modello: le risposte vere e proprie non sono state ancora generate.

Scelta del modello

Dopo di che ho scelto il modello su cui fare tutti gli esperimenti. Il modello che ho scelto è Claude Sonnet 5, perché è abbastanza capace per rispondere a domande brevi basandosi sul contesto e non serve usare un modello più pesante come Opus.

Modello: Claude Sonnet 5

ID: `claude-sonnet-5`

Effort: medium

Strumenti e ricerca web: disattivati

Conversazioni: una nuova esecuzione indipendente per ogni domanda

Prompt: sempre identico, cambia soltanto il contesto disponibile

Il modello non rappresenta una variabile dell'esperimento, perché il confronto riguarda il diverso perimetro della memoria disponibile. Ho fatto realizzare un runner che esegue ogni input separatamente, senza mantenere la conversazione precedente e senza permettere al modello di consultare i file del progetto. Il runner registra anche il modello effettivamente utilizzato e gli eventuali errori.

Ho svolto uno smoke test sulla domanda SC02-Q6 nelle quattro modalità. Il primo tentativo non è partito perché la CLI di Claude non era autenticata. Dopo aver effettuato il login, le quattro chiamate sono state completate correttamente con Claude Sonnet 5. C0 ha riconosciuto che mancavano le informazioni; C1 ha individuato il test pendente ma non il limite temporale; C2 ha fornito una risposta generica perché il retrieval non aveva recuperato la regola necessaria; con lo storico completo il modello ha indicato correttamente il limite di 15 minuti. Questo piccolo test ha mostrato concretamente la differenza tra un errore di retrieval e un errore di generazione. Non rappresenta ancora il risultato finale, perché devono essere generate e valutate tutte le 56 risposte.

Generazione delle risposte

Ho utilizzato Claude Sonnet 5 per generare le risposte dell'esperimento. Per ognuna delle 14 domande ho provato quattro situazioni: senza memoria, con memoria locale, con memoria condivisa e con tutto lo storico. In totale sono state generate 56 risposte.

Ogni domanda è stata eseguita separatamente, in modo che le risposte precedenti non potessero influenzare quelle successive. Claude ha ricevuto soltanto la domanda e le informazioni disponibili nella relativa condizione, senza conoscere le risposte dell'oracle.

Tutte le 56 prove sono state completate senza errori. In questo momento ho quindi le risposte del modello, ma devo ancora confrontarle con l'oracle per stabilire quali sono corrette e capire le cause degli eventuali errori.

Possibili sviluppi futuri

Una volta concluso il primo esperimento, se avrò tempo potrò aggiungere altre prove o architetture. Queste estensioni non sono necessarie per completare il progetto attuale, ma potrebbero permettermi di capire meglio quali tipi di memoria funzionano nei diversi casi. Le idee derivano dai paper studiati e riassunti nella revisione iniziale.

1. Potrei espandere il dataset della cybersecurity con roba ancora più interessante
2. Potrei confrontare il Turn-level RAG con un Fact-based RAG. Invece di conservare e recuperare i messaggi originali, il sistema trasformerebbe ogni messaggio in uno o più fatti brevi. In questo modo potrei verificare se una memoria più sintetica facilita il retrieval. Questa idea deriva soprattutto da LoCoMo, che utilizza osservazioni strutturate, da LongMemEval, che propone l'estrazione dei fatti, e da Mem0, che conserva informazioni sintetiche.
3. Potrei aggiungere un meccanismo per gestire le informazioni aggiornate e obsolete. Quando arriva una nuova informazione, il sistema dovrebbe capire se deve aggiungerla, aggiornare un fatto precedente, renderlo obsoleto oppure non modificare la memoria. Questa idea deriva principalmente da Mem0, che utilizza le operazioni ADD, UPDATE, DELETE e NOOP.
4. Potrei arricchire le chiavi utilizzate dal retrieval. Il messaggio originale rimarrebbe invariato, ma verrebbe associato a una descrizione più diretta dei fatti contenuti. Questo potrebbe aiutare a recuperare informazioni espresse con parole diverse da quelle presenti nella domanda. Questa idea è ripresa da LongMemEval, che propone di espandere le chiavi della memoria usando fatti estratti.
5. Potrei aggiungere informazioni temporali al retrieval, dando maggiore importanza alle decisioni più recenti e distinguendo le informazioni ancora valide da quelle superate. Questa idea deriva da LongMemEval, che utilizza l'espansione temporale delle domande, e da Mem0⁹, che rappresenta anche i cambiamenti temporali e le relazioni obsolete.
6. Potrei modificare il modo in cui il modello legge i messaggi recuperati. Prima di rispondere, il modello potrebbe controllare quali evidenze sono utili, se le informazioni sono sufficienti e se sono presenti dati obsoleti. Questa idea deriva da LongMemEval, che utilizza una strategia di lettura strutturata chiamata Chain-of-Note.
7. Potrei combinare la similarità con altri criteri, come la recenza e l'importanza di un ricordo. In questo modo il retrieval non dipenderebbe soltanto dalle parole in comune con la domanda. Questa idea deriva da Generative Agents, dove il recupero combina rilevanza, recenza e importanza.
8. Come estensione più complessa, potrei utilizzare una memoria gerarchica, separando contesto corrente, cronologia recente e archivio a lungo termine. Questa idea deriva da MemGPT, che organizza la memoria in diversi livelli ispirandosi alla memoria virtuale dei sistemi operativi.
9. Potrei infine sperimentare una memoria a grafo, nella quale decisioni, attività ed eventi sono rappresentati come nodi collegati da relazioni. Questa soluzione potrebbe aiutare nelle domande che richiedono collegamenti tra sessioni diverse, ma sarebbe più difficile da costruire e valutare. Questa idea deriva da Mem0⁹, che utilizza un knowledge graph, e da A-MEM, che collega dinamicamente note e ricordi.

Se aggiungerò una di queste estensioni, dovrò modificare una sola caratteristica alla volta e mantenere

invariati modello, domande, prompt e criteri di valutazione. Questo principio deriva soprattutto da LongMemEval e dalla survey sui meccanismi di memoria degli agenti LLM, che suggeriscono di separare le componenti e valutarle in modo controllato.

Giorno 4 (31/08/2026)

Dopo aver utilizzato Claude Code per costruire la pipeline sperimentale, ho eseguito con Claude Sonnet 5 la generazione delle 56 risposte. Successivamente ho chiesto a ChatGPT di confrontare queste risposte con l'oracle manuale e di proporre una prima classificazione. ChatGPT ha verificato la presenza dei fatti obbligatori, le astensioni, l'eventuale uso di informazioni obsolete e la presenza di affermazioni non supportate. Dal controllo sono risultate 28 risposte complete, 26 astensioni corrette, una risposta parziale e una risposta errata. I due problemi riguardano entrambi la domanda SC02-Q6. In C1 Claude non disponeva delle informazioni necessarie, ma ha provato comunque a rispondere invece di astenersi. In C2 l'informazione era presente nella memoria accessibile, ma il retrieval non ha recuperato il messaggio contenente la regola dei 15 minuti; di conseguenza Claude ha prodotto una risposta soltanto parziale. ChatGPT ha svolto una prima classificazione assistita, ma la decisione finale sulle valutazioni e sull'interpretazione dei risultati rimane sotto il mio controllo.

Ho chiesto a Claude Code di realizzare uno script per calcolare automaticamente le metriche a partire dalle 56 risposte già classificate. Gli ho specificato di non modificare scenari, oracle, retrieval o risposte generate e di considerare C0, C1 e C2 come le sole condizioni principali, mantenendo `FULL_HISTORY` come controllo separato.

Claude ha calcolato, per ogni condizione, quante domande erano raggiungibili, quante evidenze erano state recuperate, quante risposte erano complete e quante astensioni erano corrette. Ha poi esportato i risultati sia in un file JSON sia in un documento leggibile e ha aggiunto dei test automatici.

Successivamente ho chiesto a ChatGPT di controllare il lavoro svolto da Claude. ChatGPT ha verificato le formule, la separazione di `FULL_HISTORY`, la coerenza dei risultati e il corretto funzionamento dello script. Ha inoltre rigenerato i file per controllare che i risultati fossero deterministici ed eseguito tutti gli 84 test, che sono stati superati.

Da questo passaggio è emerso che, quando il retrieval ha recuperato tutte le informazioni necessarie, Claude ha sempre prodotto una risposta completa. L'unica risposta parziale in C2 è dipesa dal mancato recupero del messaggio contenente la regola dei 15 minuti.

Successivamente ho chiesto a Claude Code di ricostruire le tracce dei due problemi, confrontando il perimetro accessibile, i messaggi recuperati e le evidenze richieste dall'oracle. ChatGPT ha poi controllato l'analisi e la coerenza delle cause individuate. Nel caso SC02-Q6/C1, la causa principale è il perimetro limitato della memoria, perché la regola dei 15 minuti non era accessibile. È presente anche un errore secondario di risposta, perché Claude avrebbe dovuto astenersi ma ha provato comunque a rispondere. Nel caso SC02-Q6/C2, invece, l'informazione era accessibile ma non è stata recuperata. Il problema è quindi nel retrieval e non nella domanda, nell'oracle o nella capacità del modello di utilizzare l'informazione. Ho deciso di conservare entrambi gli errori come risultati reali del pilot, senza modificare le domande per migliorare artificialmente i risultati.

Giorno 5 (03/09/2026)

Ricevimento con il relatore

Giorno 6 (04/09/2026)

Oggi ho deciso come ampliare la seconda nuova domanda: «**Come?**». Questa nuova parte ha l'obiettivo di capire come cambia il comportamento del modello quando le stesse informazioni vengono conservate e

organizzate in modi diversi.

Confronterò quindi quattro sistemi:

- **Turn-level RAG**: conserva e recupera i messaggi originali;
- **Fact-based RAG**: trasforma i messaggi in fatti espressi in forma più breve;
- **Fact-based RAG con aggiornamenti**: permette di aggiungere, modificare o eliminare un fatto, oppure di ignorare un'informazione senza modificare la memoria;
- **Graph memory**: organizza le informazioni in un grafo, rappresentando i collegamenti tra persone, eventi, decisioni e attività.

Manterrò i due scenari esistenti e ne aggiungerò altri due. Il terzo riguarderà un'infezione malware, dai primi sintomi fino alla sua corretta classificazione. Il quarto riguarderà una campagna di smishing e social engineering, con collegamenti tra SMS, URL, un dipendente, un account e gli eventi successivi.

Il confronto sarà progressivo: le architetture non verranno applicate tutte a ogni scenario. Nel primo utilizzerò il Turn-level RAG; nel secondo confronterò il Turn-level RAG con il Fact-based RAG; nel terzo confronterò il Fact-based RAG con la sua versione dotata di aggiornamenti; nel quarto confronterò quest'ultima con la Graph memory. La condizione `FULL_HISTORY` sarà presente in tutti gli scenari come riferimento.

Il percorso seguirà quindi questa sequenza:

messaggi originali → fatti brevi → fatti aggiornabili → fatti collegati in un grafo

Claude Sonnet 5 rimarrà il modello utilizzato in tutte le prove. All'interno di ciascun confronto varierà soltanto il modo in cui la memoria viene costruita e fornita al modello. Le risposte continueranno a essere valutate mediante il confronto con l'oracle e il controllo manuale.

La nuova fase prevede complessivamente 77 prove, comprese quelle nella condizione `FULL_HISTORY`. Al momento ho completato la parte relativa a «**Quanto?**» e definito il piano di lavoro per la parte relativa a «**Come?**». Restano da costruire i due nuovi scenari, implementare le nuove architetture, eseguire le prove e analizzare i risultati.

Il prossimo passo è controllare gli oracle e fare la prima prova reale `T/F/FULL_HISTORY` su SC02. Poi esamineremo i fatti estratti e le risposte prima di procedere con SC03 e SC04. Le architetture `T/F/U/G` sono implementate; restano da verificare con Claude reale. Le metriche aggregate RQ2 e il congelamento del protocollo vengono dopo.

Giorno 7 (05/09/2026)

Oggi mi sono concentrato soprattutto sul consolidamento dell'implementazione, passando dalle verifiche offline alle prove reali con Claude.

Queste esecuzioni hanno fatto emergere problemi che i dati di prova non mostravano: riferimenti errati negli aggiornamenti, informazioni che non venivano conservate in memoria e fatti presenti ma non recuperati. Ho quindi lavorato sulla gestione delle operazioni rifiutate e verificato gli effetti fino alle risposte generate.

L'obiettivo è avere una base più affidabile prima di introdurre altre estensioni. Questo non garantisce che tutte le risposte saranno corrette, ma mi permette di verificare che le prove vengano eseguite correttamente e di capire dove nascono gli eventuali errori.

Mentre Claude mi consolidava il lavoro, come prossima idea ho pensato di effettuare la seguente: Come estensione più complessa, potrei utilizzare una memoria gerarchica, separando contesto corrente, cronologia recente e archivio a lungo termine. Questa idea deriva da MemGPT, che organizza la memoria in diversi livelli ispirandosi alla memoria virtuale dei sistemi operativi. In questo caso sarebbe un adattamento semplificato ispirato a MemGPT

Per la progettazione della memoria gerarchica la domanda è: A parità di informazioni disponibili e di budget di contesto, separare memoria recente e archivio aiuta a continuare un'attività distribuita su conversazioni lunghe?

Non assumiamo che la gerarchia migliori le risposte: potrebbe aiutare a conservare il contesto recente, ma anche sottrarre spazio a informazioni vecchie decisive.

Giorno 8 (06/09/2026)

Oggi ho implementato la memoria gerarchica con l'aiuto di Claude. Ho poi confrontato Turn-level RAG, Fact-based RAG con aggiornamenti e memoria gerarchica nell'ultimo scenario. Ho anche confrontato Turn-level RAG, Fact-based RAG con aggiornamenti e Graph-based Memory nello scenario precedente, per vedere come le diverse architetture recuperano le informazioni e come questo influisce sulle risposte.